

JOB PORTAL MANAGEMENT SYSTEM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

CHERIPALLY SURYA SRINIDHI (VTU24957)

10211CS224

Full Stack Application Development

Slot : E1



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled “Job Portal Management System” by Cheripally Surya Srinidhi (VTU24957) has been carried out during the Winter Semester of the Academic Year 2025–2026 (WS2526).

Signature of Faculty

Dr.S.Hemamalini

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr. M.Sumithra

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Cheripally Surya Srinidhi

Date: 06/05/2026

ABSTRACT

The Job Portal System is a web-based application designed to simplify and streamline the recruitment process by connecting job seekers with employers on a single platform. The system provides an intuitive interface where users can search and browse job opportunities based on keywords, location, and categories. Job seekers can register, create profiles, and explore detailed job descriptions, while employers can register, post job openings, and manage listings efficiently.

The application follows a layered architecture consisting of a frontend interface, a backend service layer, and a relational database. The frontend provides a user-friendly experience, enabling seamless navigation across features such as job search, job details, and authentication. The backend handles business logic, including user authentication, job management, and data processing through RESTful APIs. The database stores essential information such as user details, job postings, and application records. The system supports role-based access control, distinguishing between job seekers and employers to provide tailored functionalities. This improves usability and ensures secure access to features. Additionally, the platform is designed to be scalable and extensible, allowing future enhancements such as resume uploads, job application tracking, and notification systems.

Overall, the Job Portal System aims to enhance the efficiency of the hiring process by reducing manual effort, improving accessibility, and providing a centralized platform for recruitment activities.

Keywords: Job Portal, Web Application, Recruitment System, Job Search, Employer Dashboard, User Authentication, REST API, Full Stack Development, Database Management, Role-Based Access Control

LIST OF FIGURES

1.1	Framework	3
3.1	Backend Execution Output	10
4.1	MVC Architecture of Job Portal System	15
4.2	Data Flow Diagram	16
5.1	Home Screen	21
5.2	Browse Jobs Output Screen	22
5.3	Employer Dashboard Output Screen	23
5.4	Student Dashboard Output Screen	24
5.5	Chatbot	25

LIST OF TABLES

5.1	Comparison of Existing Systems and Proposed System	20
7.1	Mapping of Project to Complex Engineering Problem (CEP)	33
7.2	Mapping of Project to Complex Engineering Activities (CEA) . . .	34
7.3	SDG Mapping for the Project	34

LIST OF ACRONYMS AND ABBREVIATIONS

MVC	Model View Controller
API	Application Programming Interface
JPA	Java Persistence API
JDBC	Java Database Connectivity
SQL	Structured Query Language
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
HTTP	HyperText Transfer Protocol
DBMS	Database Management System

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	1
1.3 Scope of the Project	2
1.4 Framework	2
2 SURVEY OF SIMILAR APPLICATION IN MARKET	4
2.1 LinkedIn	4
2.2 Naukri	5
2.3 Indeed	5
2.4 Comparison with Proposed System	6
3 FRAMEWORK DESCRIPTION	7
3.1 Frontend Implementation	7
3.2 Backend Implementation	8
3.3 Database Implementation	11

4	METHODOLOGIES	14
4.1	Project Architecture	14
4.2	DataFlow Diagram	15
4.3	Module 1: User Management	17
4.4	Module 2: Job Management	17
4.5	Module 3: Application Management	17
5	RESULTS AND DISCUSSIONS	19
6	CONCLUSION AND FUTURE ENHANCEMENTS	26
6.1	Conclusion	26
6.2	Future Enhancements	26
6.3	SOURCE CODE IMPLEMENTATION	28
6.3.1	Home Page	28
6.3.2	Login Page	30
6.3.3	Dashboard	31
	References	32
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	33
7.1	Mapping of the Project to Complex Engineering Problem (CEP) . .	33
7.2	Mapping of the Project to Complex Engineering Activities (CEA) .	34
7.3	SDG Mapping for the Project	34
	References	34

Chapter 1

INTRODUCTION

1.1 Introduction

The Job Portal System is a web-based application developed to simplify and enhance the recruitment process by connecting job seekers with employers on a single platform. It provides an efficient and user-friendly interface where job seekers can search for jobs based on keywords, location, and categories, while employers can post and manage job listings. The system reduces the gap between candidates and recruiters by offering a centralized and accessible solution for job searching and hiring. This application provides a centralized platform where job seekers can search and apply for jobs, while employers can post job vacancies and manage applications effectively. The system ensures seamless communication between users and improves the overall recruitment process. By automating various tasks, the application reduces manual effort and enhances efficiency. The project follows the Model-View-Controller (MVC) architecture, which ensures proper separation of concerns between different layers of the application. The frontend is designed using Thymeleaf templates, the backend logic is implemented using Spring Boot, and MySQL is used for storing and managing data.

1.2 Aim of the Project

The main aim of this project is to design and develop an efficient job portal system that streamlines the recruitment process. It focuses on enabling job seekers to easily

find suitable job opportunities and allowing employers to efficiently post and manage job listings. The system aims to improve accessibility, reduce manual effort, and provide a secure platform for both users.

1.3 Scope of the Project

The scope of this project includes the development of a web-based platform that supports user registration, login, job search, job listing, and job detail viewing. It provides role-based access for job seekers and employers. The system can be extended in the future to include features such as job applications, resume uploads, notifications, and advanced filtering options.

1.4 Framework

The system is developed using a layered architecture consisting of a frontend, backend, and database. The frontend is built using web technologies such as HTML, CSS, and JavaScript to provide an interactive user interface. The backend is developed using a framework such as Spring Boot to handle business logic, authentication, and API services. The database, such as MySQL or PostgreSQL, is used to store user information, job listings, and related data securely.

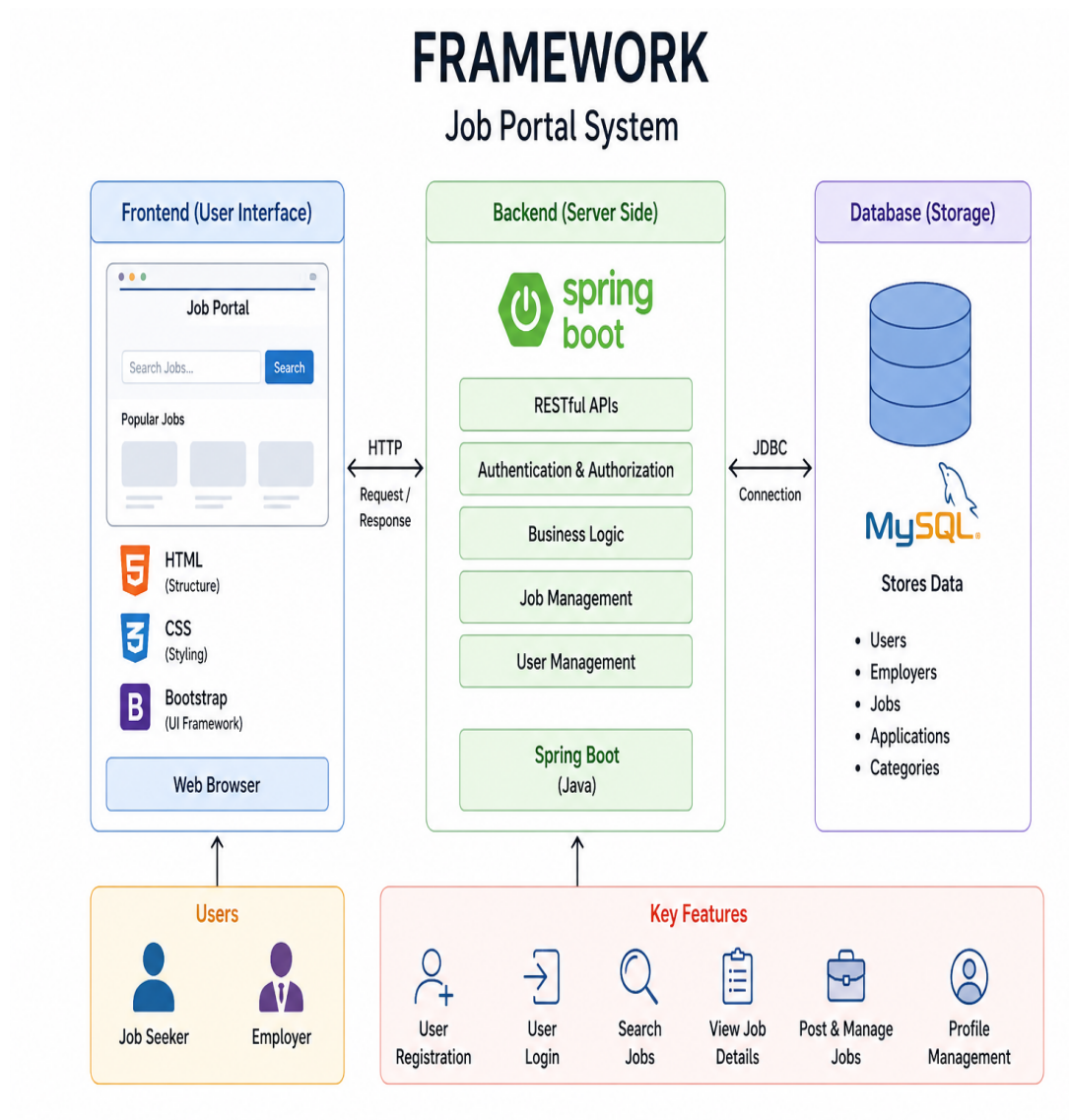


Figure 1.1: Framework

Figure 1.1 shows the framework of the job portal system, illustrating a multi-layered architecture that consists of the Frontend (User Interface), Backend (Server Side), and Database (Storage). This tiered design facilitates seamless interaction between job seekers and employers through RESTful APIs and JDBC connectivity, ensuring modularity, scalability, and secure data management across the platform.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

The job portal system developed in this project is inspired by several existing applications widely used in the recruitment industry. These platforms provide various features that help job seekers find opportunities and enable employers to hire suitable candidates efficiently. Some of the popular job portal applications available in the market are:

2.1 LinkedIn

LinkedIn is one of the most widely used professional networking platforms. It allows users to create professional profiles, connect with industry experts, and apply for jobs. Recruiters can post job openings and search for candidates based on their skills and experience.

Advantages:

- Strong professional networking features
- Job recommendations based on user profiles
- Easy communication between recruiters and candidates

Limitations:

- Complex interface for new users
- Many features require premium subscription

2.2 Naukri

Naukri is a popular job portal widely used in India. It provides a platform for job seekers to upload resumes and apply for jobs, while employers can search for suitable candidates.

Advantages:

- Large database of job listings
- Resume upload and profile management
- Simple job search functionality

Limitations:

- Limited personalization of job recommendations
- Frequent notifications may be distracting

2.3 Indeed

Indeed is a global job search platform that aggregates job listings from various sources. It allows users to search for jobs based on location, salary, and job type.

Advantages:

- Wide range of job listings from multiple sources
- Easy-to-use interface
- Free access for job seekers

Limitations:

- Duplicate job postings may appear
- Limited direct interaction with employers

2.4 Comparison with Proposed System

The proposed Job Portal Application aims to overcome some of the limitations of existing systems by providing a simple, user-friendly interface and essential functionalities without unnecessary complexity. Unlike existing platforms, the system focuses on ease of use, efficient job management, and streamlined communication between users.

Key Improvements:

- Simplified user interface
- Focused features for academic-level implementation
- Efficient database management using MySQL
- Secure authentication using Spring Boot

Thus, the proposed system provides a balanced solution by combining essential features of existing applications while maintaining simplicity and efficiency.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of this application is tightly integrated with the backend using server-side rendering. The environment setup for the frontend requires:

- **Web Browser:** Any modern web browser such as Google Chrome, Mozilla Firefox, or Microsoft Edge to render and test the application interface.
- **Template Engine:** Thymeleaf is used as the primary template engine, requiring the `spring-boot-starter-thymeleaf` dependency to dynamically generate HTML views.
- **Third-Party Libraries:** External CDNs are used for UI features, including Google Fonts (Inter) for typography and Leaflet.js / OpenStreetMap for map integration. These are loaded dynamically without local installation.

3.1.2 Structure of the Project

The frontend components are organized within the `src/main/resources` directory:

- **templates/:** Contains all Thymeleaf HTML files
 - **employer/:** dashboard.html, post-job.html, applicants.html
 - **student/:** dashboard.html
 - **job/:** list.html, details.html

- **fragments/:** header.html, footer.html
- **static/css/:** Contains `style.css` with custom styling, glassmorphism effects, and responsive design.

3.1.3 Execution Procedure

Since the frontend uses server-side rendering, it does not run independently.

- User navigates to a URL (e.g., `http://localhost:8080/jobs`)
- Browser sends an HTTP request to the backend
- Backend controller processes the request and injects data into Thymeleaf templates
- HTML is rendered and sent back to the browser
- Browser applies CSS and executes JavaScript

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is developed using Spring Boot. The setup requires:

- **JDK:** Java 17 installed and configured
- **IDE:** Spring Tools for Eclipse
- **Build Tool:** Apache Maven
- **Dependencies:**
 - `spring-boot-starter-web`
 - `spring-boot-starter-security`

- `spring-boot-starter-data-jpa`
- `spring-boot-starter-mail`

3.2.2 Structure of the Project

The backend follows an MVC architecture inside `src/main/java/com/example/jobportal`

- **model/:** Entity classes (User, Job, Application, Role)
- **repository/:** JPA repositories for database operations
- **service/:** Business logic layer (UserService, JobService, etc.)
- **controller/:** Handles HTTP requests (AuthController, EmployerController, etc.)
- **config/:** Security configuration and data initialization

3.2.3 Execution Procedure The execution of the backend system is carried out using the following steps:

- Open the project folder in an IDE such as Visual Studio Code, Eclipse, or IntelliJ IDEA.
- Ensure that the MySQL database server is running in the background.
- Verify the database configuration in the `application.properties` file.
- Build the project using Maven by running `mvn clean install`.
- Run the Spring Boot application using:
 - The main class (`JobportalApplication.java`), or
 - The command `mvn spring-boot:run`
- The embedded Apache Tomcat server starts and listens on port 8080.

- Open a browser and access `http://localhost:8080` to interact with the application.

applicant_name	job_title	status	applied_at
Surya Srinidhi	Data Engineer	APPLIED	2026-05-05
Surya Srinidhi	Technical Writer	APPLIED	2026-05-05

Figure 3.1: Backend Execution Output

Figure 3.1 shows the Backend Execution Output, displaying a list of job applications. The table records data including the applicant name, job title, status (showing "APPLIED"), and the timestamp of the application, confirming successful database connectivity and backend processing.

3.3 Database Implementation

3.3.1 Database Configuration

The system uses MySQL as the database. Configuration is defined in `application.properties`

- **Database URL:** `jdbc:mysql://localhost:3306/jobportaldb`
- **Credentials:** MySQL username and password
- **Hibernate Dialect:** `MySQLDialect`
- **DDL Auto:** `update` (auto schema update)

The database automatically creates and updates tables based on entity classes, ensuring smooth development and data persistence.

3.1.2 Schema Structure

The schema for the Job Portal System is designed using an Entity-Relationship (ER) model implemented through JPA/Hibernate. The database follows a highly relational structure, utilizing primary keys and foreign keys to establish strong relationships between entities and ensure data integrity. The core architecture is based on a **one-to-many relationship** between employers and jobs, where a single employer can post multiple job listings. Additionally, a **many-to-many relationship** exists between students and jobs, which is managed through an intermediary entity called **Application**. This allows multiple students to apply for multiple jobs efficiently. The system uses the **InnoDB storage engine**, which provides full ACID (Atomicity, Consistency, Isolation, Durability) compliance. This ensures that critical operations, such as job applications, are processed reliably and consistently without data loss or corruption.

3.1.3 Tables created

Hibernate automatically generates the following core tables in the `jobportaldb` database based on the defined Java Entity models:

1. Users Table

This table stores authentication credentials and profile information for all users of the platform.

- **id (Primary Key, BIGINT):** Auto-incremented unique identifier for each user.
- **dept (VARCHAR):** Department of the user (e.g., Computer Science, IT, Mechanical).
- **email (VARCHAR, Unique):** Email address used for communication and login.
- **fullname (VARCHAR, Unique):** Login name chosen by the user.
- **password (VARCHAR):** Securely hashed password using BCrypt.
- **resume_file_path (VARCHAR):** File path of the uploaded resume (for students).
- **role (ENUM):** Defines user role (STUDENT, EMPLOYER, ADMIN).
- **year_of_study (INT):** Current academic year of the student (e.g., 1, 2, 3, 4).

2. Jobs Table

This table contains all job postings created by employers.

- **created_at (DATE/DATETIME):** Date and time when the job was posted.
- **latitude (DOUBLE):** Latitude coordinate of the job location.

- **longitude (DOUBLE):** Longitude coordinate of the job location.
- **employer_id (Foreign Key, BIGINT):** References the employer in the Users table.
- **id (Primary Key, BIGINT):** Auto-incremented unique identifier for each job.
- **description (VARCHAR):** Detailed job responsibilities.
- **location_name (VARCHAR):** Job location or remote status.
- **salary (VARCHAR):** Offered salary package.
- **skills_required (VARCHAR):** Required skills for the role.
- **title (VARCHAR):** Job title (e.g., Senior Java Developer).

3. Applications Table

This table acts as a join table connecting students with jobs they have applied for and tracks application status.

- **applied_at (DATETIME):** Timestamp of application submission.
- **id (Primary Key, BIGINT):** Unique identifier for each application.
- **job_id (Foreign Key, BIGINT):** References the job in the Jobs table.
- **user_id (Foreign Key, BIGINT):** References the student in the Users table.
- **resume_file_path (VARCHAR):** Stores the file path of the uploaded resume for the student.
- **status (ENUM):** Current status (PENDING, SHORTLISTED, REJECTED).

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The project follows the Model-View-Controller (MVC) architecture, which separates the application into three main components: Model, View, and Controller. This separation improves maintainability, scalability, and code organization.

- **Model:** Represents the data layer, including entity classes and database interactions using JPA.
- **View:** Implemented using Thymeleaf templates to display dynamic content to users.
- **Controller:** Handles incoming requests, processes them using services, and returns appropriate responses.

The architecture ensures a smooth flow of data between the user interface and the database. The overall structure of the system is shown in [Figure4.1], which illustrates the interaction between different layers of the application.

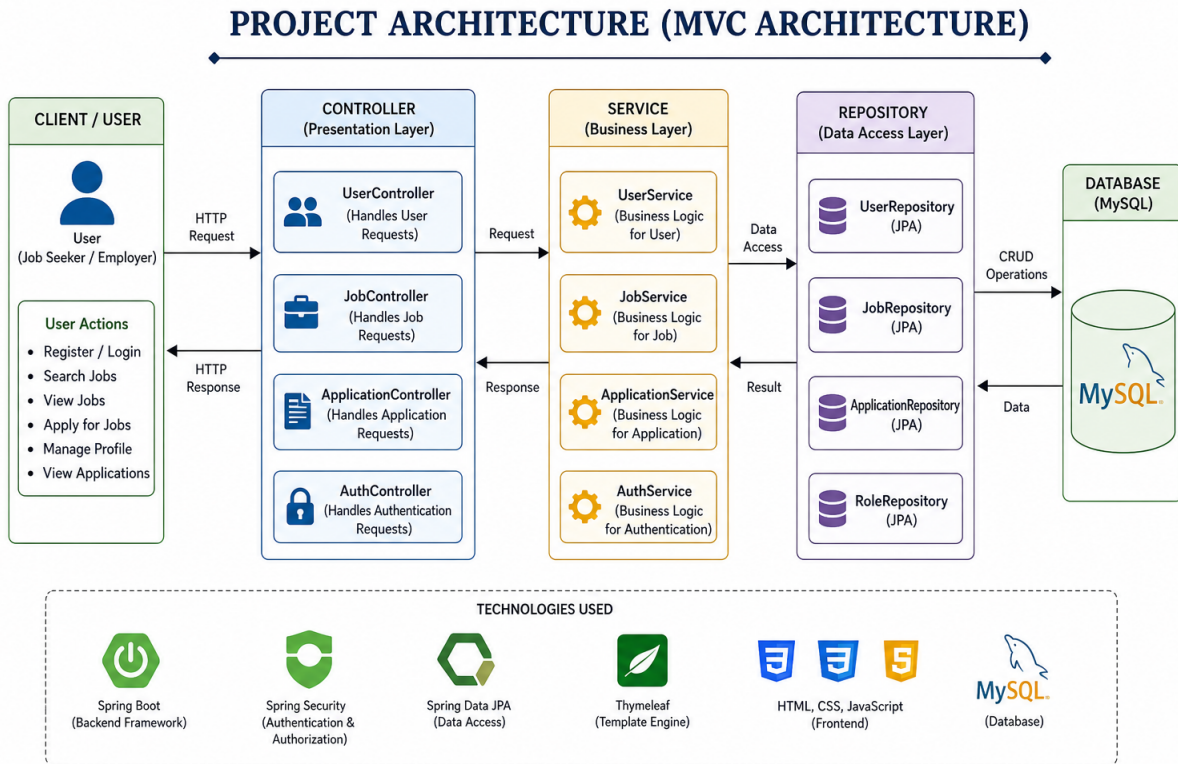


Figure 4.1: MVC Architecture of Job Portal System

Figure 4.1 shows the system architecture of the job portal, featuring a multi-layered MVC structure composed of the Presentation (Controllers), Business (Services), and Data Access (Repositories) layers. This design ensures modularity and scalability, enabling seamless communication between the Thymeleaf frontend and the MySQL database via Spring Data JPA.

4.2 DataFlow Diagram

The Data Flow Diagram (DFD) represents how data moves through the system. It shows the interaction between users, the application, and the database.

- Users provide input through the frontend (login, registration, job search).
- The controller processes the request and sends it to the service layer.
- The service layer interacts with the repository to access the database.

- The database returns the required data.
- The result is displayed to the user through the frontend.

The data movement within the system is illustrated in [Figure4.2], showing how information flows between different components during execution.

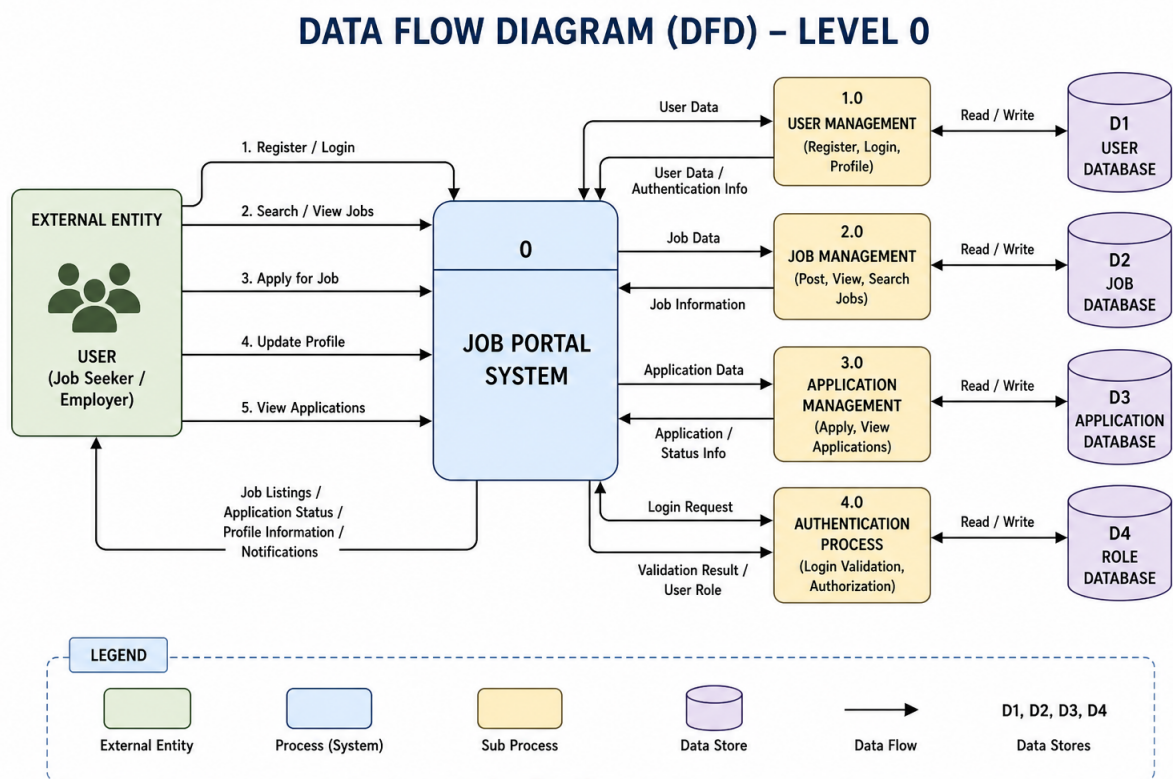


Figure 4.2: Data Flow Diagram

Figure 4.2 shows the Level 0 Data Flow Diagram (DFD) of the job portal, illustrating the interaction between the User (Job Seeker/Employer) and the core system. It highlights the flow of data for key processes such as registration, job searching, and application management, while showing how information is exchanged with the database to maintain user profiles and job records.

4.3 Module 1: User Management

This module handles user-related functionalities such as registration, login, and profile management.

- User registration with basic details
- Secure login authentication
- Profile viewing and updating

4.4 Module 2: Job Management

This module is responsible for managing job postings.

- Employers can post job vacancies
- Job details such as title, description, and company information are stored
- Users can browse available jobs

4.5 Module 3: Application Management

This module allows users to apply for jobs and manage applications.

- Users can apply for jobs
- Applications are stored in the database
- Employers can view applicants

Advantages of this Project

The proposed system provides several advantages:

a) User-Friendly Interface: The application provides a simple and easy-to-use interface for both job seekers and employers.

b) Efficient Data Management: All data is stored and managed using MySQL, ensuring quick access and reliability.

c) Scalable Architecture: The use of Spring Boot and MVC architecture allows the system to be easily extended with additional features.

Disadvantages

- Limited advanced features such as AI-based job recommendations.
- No email notification system for job updates.
- Requires internet connection for access.

Chapter 5

RESULTS AND DISCUSSIONS

The developed Job Portal Application was successfully implemented using Spring Boot, Thymeleaf, and MySQL. The system was tested for various functionalities including user registration, login, job posting, and job application[table 5.1].

The application provides a smooth and user-friendly interface, allowing users to easily interact with the system. The backend processes requests efficiently, and the database ensures secure and reliable data storage.

Observed Results:

- Users are able to register and log in successfully.
- Employers can post job vacancies without errors.
- Job seekers can search and apply for jobs.
- Data is stored and retrieved correctly from MySQL database.
- The system performs efficiently with minimal response time.

Table 5.1: Comparison of Existing Systems and Proposed System

Features	Existing Systems	Proposed System
User Interface	Complex	Simple and User-Friendly
Cost	Paid Features	Free to Use
Job Search	Available	Available
Job Application	Available	Available
Customization	Limited	Flexible
Performance	Moderate	Efficient

Table 5.1 presents a comparative analysis between existing systems and the proposed solution based on metrics like User Interface, Cost, and Performance. The results demonstrate that the proposed system is more user-friendly, cost-effective, and flexible, providing an efficient alternative for job seekers and employers.

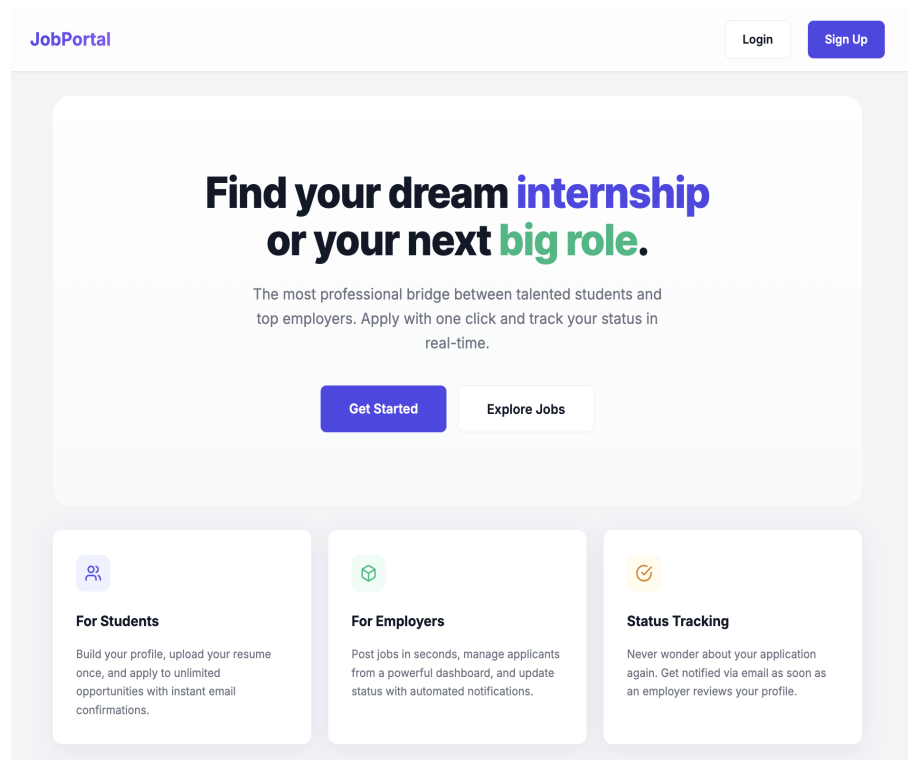


Figure 5.1: Home Screen

Figure 5.1 illustrates the Home Screen of the job portal, showcasing a modern and user-friendly interface. It features a central call-to-action for users to find internships and roles, with distinct sections for Students, Employers, and Status Tracking. This landing page serves as the primary entry point, providing seamless navigation to explore jobs or register via the "Get Started" and "Login" options.

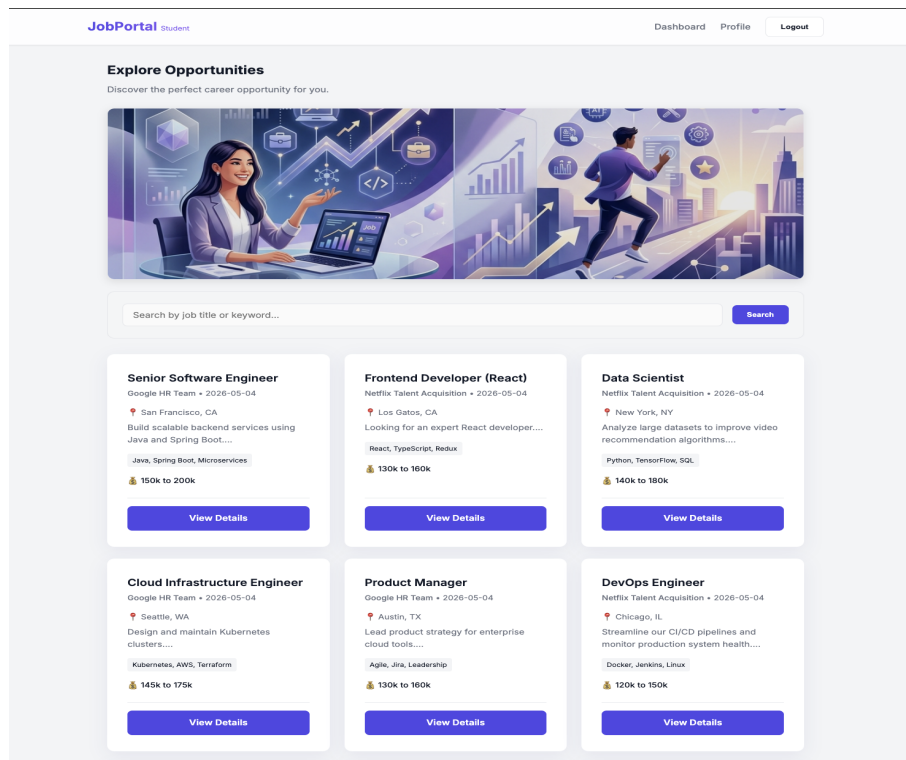


Figure 5.2: Browse Jobs Output Screen

Figure 5.2 showcases the Browse Jobs Interface, which acts as the central marketplace of the platform. This screen utilizes a dynamic filtering mechanism that allows job seekers to parse through various roles, such as Software Engineering or Data Science, based on real-time data. Each job card is rendered with essential metadata—including salary brackets, geographic location, and company branding—enabling users to make informed decisions at a glance. Technically, this view is supported by Spring Data JPA pagination, ensuring that even with a high volume of job postings, the interface remains performant and responsive to user queries.

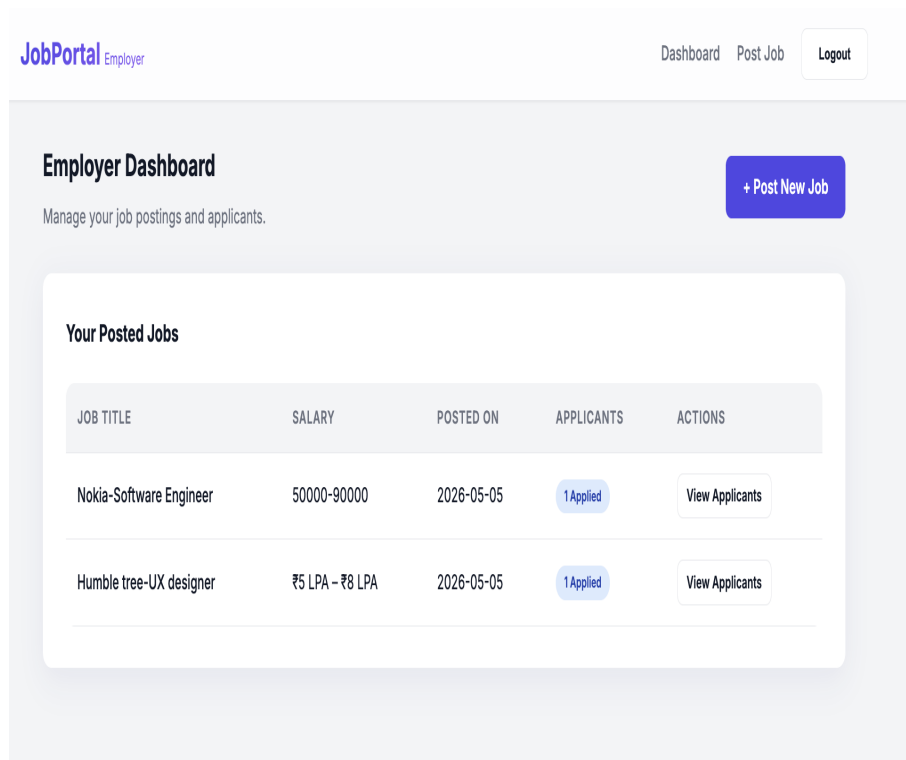


Figure 5.3: Employer Dashboard Output Screen

Figure 5.3 illustrates the Employer Dashboard, a specialized administrative module designed for talent management. This interface provides recruiters with a high-level overview of their active postings, featuring quantitative metrics such as the total number of applicants for each role. The dashboard includes integrated functional buttons for editing, deleting, or viewing candidate profiles, facilitating a streamlined workflow for the recruitment lifecycle. By centralizing these management tools, the system minimizes administrative overhead and allows employers to track hiring progress through a single, cohesive view.

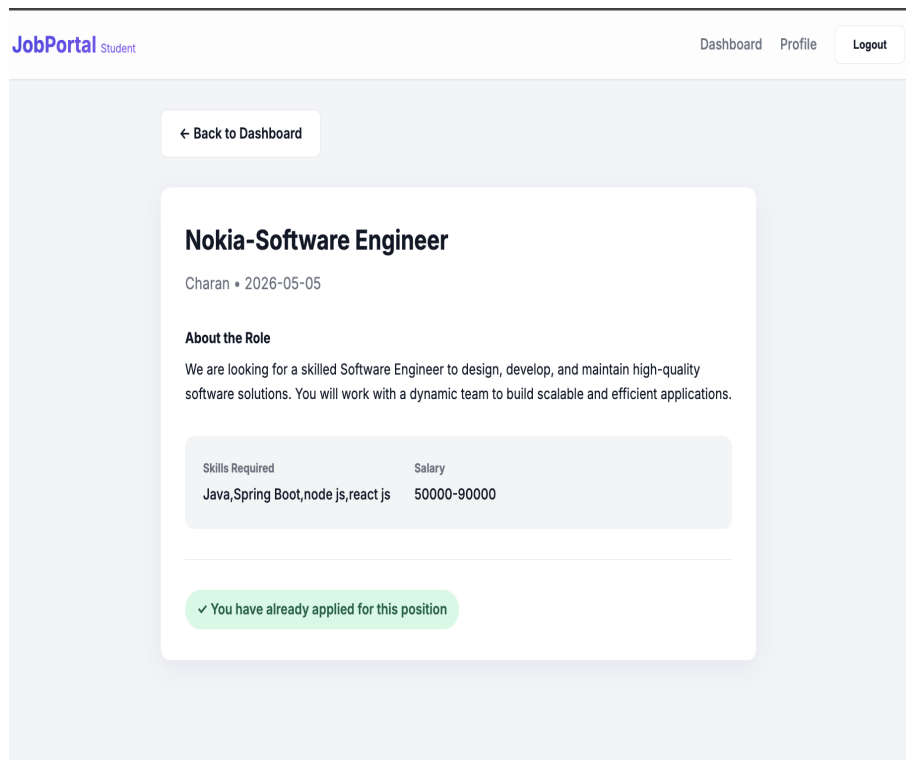


Figure 5.4: Student Dashboard Output Screen

Figure 5.4 depicts the Student Application Dashboard, which provides a transparent view of a candidate's journey. This screen displays granular details for specific roles, highlighting required technical competencies (e.g., Java, Spring Boot) and compensation packages. A critical feature shown here is the Application Status Indicator, which prevents redundant submissions by notifying the student if they have already applied for a particular position. This feedback loop is essential for maintaining data integrity and improving the overall user journey by providing immediate confirmation of backend state changes.

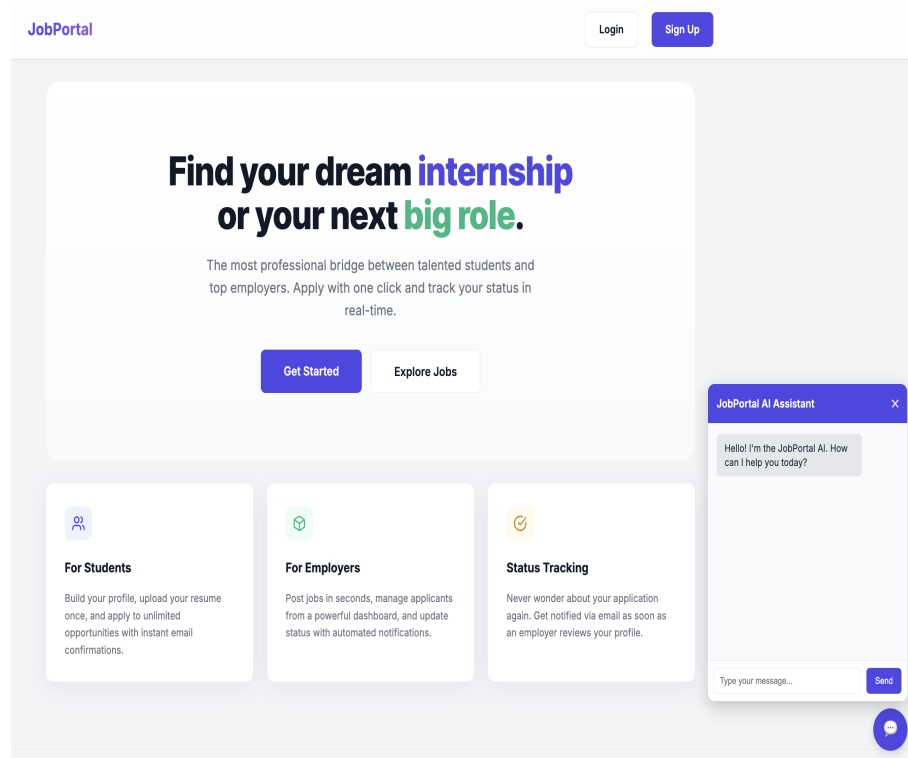


Figure 5.5: Chatbot

Figure 5.5 presents the AI-Integrated Chatbot Interface, a sophisticated support feature designed to enhance the platform's interactivity. The chatbot serves as an intelligent assistant, capable of interpreting user intent to provide instant answers regarding job availability, system navigation, and application tracking. By integrating this conversational agent, the portal provides a 24/7 support layer that significantly reduces the dependency on manual helpdesk intervention. This feature not only improves user retention but also ensures that less tech-savvy users can navigate the complex features of the portal with ease.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Job Portal Application was successfully designed and implemented using Spring Boot, Thymeleaf, and MySQL. The system provides an efficient platform for job seekers and employers to interact, reducing the complexity of traditional recruitment processes.

The application supports essential functionalities such as user registration, login authentication, job posting, and job application. The use of MVC architecture ensures a well-structured and maintainable system. The integration of frontend and backend technologies enables smooth data flow and dynamic content rendering.

Overall, the project meets its objectives by providing a simple, scalable, and user-friendly solution. It demonstrates the effective use of modern web technologies in building real-world applications.

6.2 Future Enhancements

Although the system performs well, several improvements can be made in the future to enhance its functionality:

- **Email Notifications:** Notify users about job updates and application status.
- **Resume Upload Feature:** Allow users to upload and manage resumes.

- **Advanced Search Filters:** Improve job search with filters like salary, location, and experience.
- **Admin Dashboard:** Add admin control for managing users and job postings.
- **Mobile Application:** Develop an Android/iOS version of the system.
- **AI-based Recommendations:** Suggest jobs based on user skills and preferences.

6.3 SOURCE CODE IMPLEMENTATION

6.3.1 Home Page

```
1 <!DOCTYPE html>
2 <html xmlns:th="http://www.thymeleaf.org">
3 <head th:replace="~{fragments :: head('JobPortal - Find Your Career Path')}"></head>
4 <body>
5
6 <div th:replace="~{fragments :: navbarGuest}"></div>
7
8 <main class="main-content">
9     <section class="hero-section" style="text-align: center; padding: 6rem 1rem;
10         background: linear-gradient(to bottom, #ffffff, #f9fafb); border-radius:
11         24px; margin-bottom: 4rem;">
12         <h1 style="font-size: 3.5rem; font-weight: 800; margin-bottom: 1.5rem;
13             letter-spacing: -0.025em; line-height: 1.1;">
14             Find your dream <span style="color: var(--primary);">internship </span> <br>
15             or your next <span style="color: var(--secondary);">big
16             role </span>.
17         </h1>
18         <p style="font-size: 1.25rem; color: var(--text-muted); max-width: 600px;
19             margin: 0 auto 2.5rem; line-height: 1.6;">
20             The most professional bridge between talented students and top
21             employers.
22             Apply with one click and track your status in real-time.
23         </p>
24         <div style="display: flex; gap: 1rem; justify-content: center;">
25             <a href="/register" class="btn btn-primary" style="padding: 1rem 2.5
26                 rem; font-size: 1.1rem;">Get Started </a>
27             <a href="/login" class="btn btn-secondary" style="padding: 1rem 2.5
28                 rem; font-size: 1.1rem;">Explore Jobs </a>
29         </div>
30     </section>
31
32     <div class="job-grid" style="margin-top: -2rem;">
33         <!-- Feature Card 1 -->
34         <div class="glass-card">
35             <div style="height: 48px; width: 48px; background: #EEF2FF; border-
36                 radius: 12px; display: flex; align-items: center; justify-content
37                 : center; margin-bottom: 1.5rem;">
38                 <svg width="24" height="24" viewBox="0 0 24 24" fill="none"
39                     stroke="var(--primary)" stroke-width="2" stroke-linecap="
40                     round" stroke-linejoin="round"><path d="M16 21v-2a4 4 0 0
41                     0-4-4H6a4 4 0 0 0-4 4v2"></path><circle cx="9" cy="7" r="4"
42                     ></circle><path d="M22 21v-2a4 4 0 0 0-3-3.87"></path><path d
43                     ="M16 3.13a4 4 0 0 1 0 7.75"></path></svg>
```

```

28     </div>
29     <h3 style="margin-bottom: 1rem;">For Students </h3>
30     <p style="color: var(--text-muted); font-size: 0.95rem;">Build your
        profile , upload your resume once , and apply to unlimited
        opportunities with instant email confirmations.</p>
31 </div>
32
33 <!-- Feature Card 2 -->
34 <div class="glass-card">
35     <div style="height: 48px; width: 48px; background: #ECFDF5; border-
        radius: 12px; display: flex; align-items: center; justify-content
        : center; margin-bottom: 1.5rem;">
36         <svg width="24" height="24" viewBox="0 0 24 24" fill="none"
            stroke="var(--secondary)" stroke-width="2" stroke-linecap="
            round" stroke-linejoin="round"><path d="M21 16V8a2 2 0 0
            0-1-1.731-7-4a2 2 0 0 0-2 01-7 4A2 2 0 0 0 3 8v8a2 2 0 0 0 1
            1.7317 4a2 2 0 0 0 2 017-4A2 2 0 0 0 21 16z"></path><polyline
            points="3.27 6.96 12 12.01 20.73 6.96"></polyline><line x1="
            12" y1="22.08" x2="12" y2="12"></line></svg>
37     </div>
38     <h3 style="margin-bottom: 1rem;">For Employers </h3>
39     <p style="color: var(--text-muted); font-size: 0.95rem;">Post jobs
        in seconds , manage applicants from a powerful dashboard , and
        update status with automated notifications.</p>
40 </div>
41
42 <!-- Feature Card 3 -->
43 <div class="glass-card">
44     <div style="height: 48px; width: 48px; background: #FFFBEB; border-
        radius: 12px; display: flex; align-items: center; justify-content
        : center; margin-bottom: 1.5rem;">
45         <svg width="24" height="24" viewBox="0 0 24 24" fill="none"
            stroke="#D97706" stroke-width="2" stroke-linecap="round"
            stroke-linejoin="round"><path d="M22 11.08V12a10 0 1
            1-5.93-9.14"></path><polyline points="22 4 12 14.01 9 11.01"
            ></polyline></svg>
46     </div>
47     <h3 style="margin-bottom: 1rem;">Status Tracking </h3>
48     <p style="color: var(--text-muted); font-size: 0.95rem;">Never
        wonder about your application again. Get notified via email as
        soon as an employer reviews your profile.</p>
49 </div>
50 </div>
51 </main>
52
53 <footer style="text-align: center; padding: 4rem 2rem; color: var(--text-muted);
        font-size: 0.875rem; border-top: 1px solid var(--border); margin-top: 4rem;"
        >

```

```

54     <p>&copy; 2026 JobPortal Management System. Built with Spring Boot & MySQL
      . </p>
55 </footer>
56
57 <div th:replace="~{fragments :: aiChatbot}"></div>
58 </body>
59 </html>

```

6.3.2 Login Page

```

1 <!DOCTYPE html>
2 <html xmlns:th="http://www.thymeleaf.org">
3 <head th:replace="~{fragments :: head('Login - Job Portal')}"></head>
4 <body>
5
6 <nav th:replace="~{fragments :: navbarGuest}"></nav>
7
8 <div class="main-content">
9     <div class="auth-container glass-card">
10         <h2 style="margin-bottom: 2rem; text-align: center;">Welcome Back</h2>
11
12         <div th:if="${param.error}" class="badge badge-red" style="display:
            block; margin-bottom: 1rem; padding: 0.5rem;">
13             Invalid email or password.
14         </div>
15         <div th:if="${param.logout}" class="badge badge-green" style="display:
            block; margin-bottom: 1rem; padding: 0.5rem;">
16             You have been logged out.
17         </div>
18         <div th:if="${success}" class="badge badge-green" style="display: block;
            margin-bottom: 1rem; padding: 0.5rem;" th:text="${success}">
19         </div>
20
21         <form th:action="@{/loginProcess}" method="post">
22             <div class="form-group">
23                 <label class="form-label" for="username">Email Address</label>
24                 <input type="email" id="username" name="username" class="form-
                    control" placeholder="john@example.com" required autofocus>
25             </div>
26
27             <div class="form-group">
28                 <label class="form-label" for="password">Password</label>
29                 <input type="password" id="password" name="password" class="form
                    -control" placeholder=" " required>
30             </div>
31
32             <button type="submit" class="btn btn-primary" style="width: 100%;
                margin-top: 1rem;">Sign In</button>

```

```

33     </form>
34
35     <p style="text-align: center; margin-top: 1.5rem; font-size: 0.875rem;
36         color: var(--text-muted);">
37         Don't have an account? <a href="/register">Sign up here </a>
38     </p>
39 </div>
40
41 <div th:replace="~{fragments :: aiChatbot}"></div>
42 </body>
43 </html>

```

6.3.3 Dashboard

```

1 <!DOCTYPE html>
2 <html xmlns:th="http://www.thymeleaf.org">
3 <head th:replace="~{fragments :: head('Employer Dashboard')}"></head>
4 <body>
5
6 <nav th:replace="~{fragments :: navbarEmployer}"></nav>
7
8 <div class="main-content">
9     <div class="dashboard-header">
10         <div>
11             <h2>Employer Dashboard </h2>
12             <p style="color: var(--text-muted); margin-top: 0.5rem;">Manage your
13                 job postings and applicants.</p>
14         </div>
15         <a href="/employer/job/post" class="btn btn-primary">+ Post New Job </a>
16     </div>
17
18     <div th:if="${success}" class="badge badge-green" style="display: block;
19         margin-bottom: 2rem; padding: 0.75rem;" th:text="${success}"></div>
20
21     <div class="glass-card">
22         <h3 style="margin-bottom: 1.5rem;">Your Posted Jobs </h3>
23
24         <div th:if="${#lists.isEmpty(jobs)}" style="text-align: center; color:
25             var(--text-muted); padding: 3rem;">
26             No jobs posted yet. Click the button above to post your first job!
27         </div>
28
29         <table class="modern-table" th:unless="${#lists.isEmpty(jobs)}">
30             <thead>
31                 <tr>
32                     <th>Job Title </th>
33                     <th>Salary </th>

```



```

31         <th>Posted On</th>
32         <th>Applicants </th>
33         <th>Actions </th>
34     </tr>
35 </thead>
36 <tbody>
37     <tr th:each="job : ${jobs}">
38         <td th:text="${job.title}" style="font-weight: 500;">Title </
39         td>
40         <td th:text="${job.salary}">Salary </td>
41         <td th:text="${job.createdAt}">Date </td>
42         <td>
43             <span class="badge badge-blue" th:text="${#lists.size(
44                 job.applications) + ' Applied'}"></span>
45         </td>
46         <td>
47             <a th:href="@{/employer/job/{id}/applicants(id=${job.id
48                 })}" class="btn btn-secondary" style="padding: 0.4rem
49                 0.8rem; font-size: 0.85rem;">View Applicants </a>
50         </td>
51     </tr>
52 </tbody>
53 </table>
54 </div>
55 </div>
56 <div th:replace="~{fragments :: aiChatbot}"></div>
57 </body>
58 </html>

```

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	In-depth knowledge	WK3,4,5,6	Requires Java, Spring Boot, JPA, and database design for Job Portal system.
WP2	Conflicting Requirements	Tech & non-tech conflicts	WK4,5,6	Balances security with usability for job seekers and recruiters.
WP3	Depth of Analysis	Analytical design	WK3,4,5	Includes ER design, APIs, resume upload, and application logic.
WP4	Familiarity	Partially new	WK4,8	Job matching and tracking beyond CRUD systems.
WP5	Applicable Codes	Limited standards	WK6	Custom logic for resume, filtering, and tracking system.
WP6	Stakeholder Needs	Multiple users	WK5,6,7	Supports job seekers, recruiters, and admin roles.
WP7	Interdependence	System integration	WK3,5,6	Integrates frontend, backend, DB, and security modules.

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

EA No.	Attribute	Characteristics	Justification
EA1	Resources	Diverse tech	Uses Java, Spring Boot, MySQL, HTML, CSS, JS.
EA2	Interaction	Complex modules	Handles jobs, applications, login, and uploads.
EA3	Innovation	Modern tech	Role-based dashboards, filtering, email alerts.
EA4	Society Impact	User benefit	Improves hiring and job accessibility.
EA5	Familiarity	Beyond basics	Combines tracking, resume, and admin system.

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

7.3 SDG Mapping for the Project

SDG No.	SDG Title	Relevance
SDG 8	Decent Work	Connects job seekers and employers for employment.
SDG 9	Innovation	Uses modern web technologies for recruitment.
SDG 10	Reduced Inequality	Equal job access for all users.
SDG 16	Strong Institutions	Ensures security using authentication system.

Table 7.3: SDG Mapping for the Project

References

- [1] LinkedIn Job Portal, 2024. [Online]. Available: <https://www.linkedin.com>
- [2] Naukri Job Portal, 2024. [Online]. Available: <https://www.naukri.com>
- [3] Indeed Job Portal, 2024. [Online]. Available: <https://www.indeed.com>
- [4] Glassdoor Job Portal, 2024. [Online]. Available: <https://www.glassdoor.com>
- [5] Monster Job Portal, 2024. [Online]. Available: <https://www.monster.com>
- [6] Shine Job Portal, 2024. [Online]. Available: <https://www.shine.com>
- [7] Spring Boot Documentation, VMware Inc., 2024. [Online]. Available: <https://spring.io/projects/spring-boot>
- [8] Spring Framework Documentation, VMware Inc., 2024. [Online]. Available: <https://spring.io/projects/spring-framework>
- [9] Thymeleaf Documentation, 2024. [Online]. Available: <https://www.thymeleaf.org>
- [10] MySQL Documentation, Oracle Corporation, 2024. [Online]. Available: <https://dev.mysql.com/doc/>
- [11] Oracle Java Documentation, Oracle Corporation, 2024. [Online]. Available: <https://docs.oracle.com/en/java/>
- [12] W3Schools HTML Tutorial, 2024. [Online]. Available: <https://www.w3schools.com/html/>
- [13] W3Schools CSS Tutorial, 2024. [Online]. Available: <https://www.w3schools.com/css/>

[14] W3Schools JavaScript Tutorial, 2024. [Online]. Available:

<https://www.w3schools.com/js/>

[15] GeeksforGeeks Spring Boot Guide, 2024. [Online]. Available:

<https://www.geeksforgeeks.org/spring-boot/>